

WeatherCheck

ML → DL → LLM/RAG 로 만드는 개인형 출근 비서

기상청은 날씨를 알려주지만, 내 비서는 내가 뭘 해야 할지를 알려준다.

발표자

학습 프로젝트 (ML·DL·LLM)

제출

2026-07-07

PHAS

01 ML

scikit-learn 기본기

PHAS

02 DL

LSTM 시계열 딥러닝

PHAS

03 LLM/RAG

로컬 sLLM 출근 비서

[github.com/50seok/
WeatherCheck](https://github.com/50seok/WeatherCheck)

목차 (Agenda)

01

시작하며 · 프로젝트 개요

배경 · 문제 정의 / 목표 · 차별화 전략 / 전체 아키텍처

슬라이드 3-6

02

Phase 1 · ML 날씨 예측 모델

데이터 수집 · EDA / 상관관계 / 모델 3 종 비교 / 분류 평가 · 피쳐 중요도

슬라이드 7-11

03

Phase 2 · DL 시계열 딥러닝

LSTM 도입 · 학습 구조 / ML vs DL 정확도 / 시계열 예측

슬라이드 12-15

04

Phase 3 · LLM/RAG 출근 비서 + 코드 구조 + 회고

RAG 파이프라인 / 디스코드 봇 / 코드 리뷰 4 장 / 트러블슈팅 / 성과 · 회고

슬라이드 16-25

01

BACKGROUND & STRATEGY

시작하며

왜 이 프로젝트를 시작했는지 ,
무엇을 차별화했는지 .

배경 & 문제 정의

≡ 기존 솔루션 한계

이름	한계
☐ 날씨 앱	오늘 날씨만 얘기해준다 , 해석은 내 몫
☁ 기상청 예보	정확하지만 '그래서 뭘 해야 하나' 없음
✦ 일반 날씨봇	누구에게나 똑같은 (내 체감 / 루트 무시)

→ 세 접근 모두 '정보 전달'에 머물고 , '결정 지원'은 빠져 있다 .

◆ 결론

공개 날씨 예측이 아니라 , **나의 출근 브리핑**으로 풀어야 의미 있고 차별화된다 .

FROM

공개 날씨 예측

TO

나의 출근 브리핑

목표 & 차별화 전략

핵심 목표 ~~AI~~ 맞춤 행동 지침 자동 발송

학습 목표 ML→DL→LLM 풀스택

차별화 나만의 데이터

차별화 전략 3 종

3 PILLARS

① 개인 체감 학습

SIGNATURE

피드백 버튼으로 나의 체감온도 학습 →
옷차림 조언에 반영 . 프로젝트 시그니처
기능 .

◆ FEEDBACK LOOP

② 니치 타겟

모두의 날씨 → 특정 상황 전용 비서 . 1 차 :
자전거 통근족 .

⊕ NICHE TARGET

③ 내 루틴 결합

출근지 등록 → 날씨 + 출퇴근
소요시간을 같이 브리핑 .

🕒 ROUTINE FUSION

전체 아키텍처 — 하나의 테마로 쌓아올린 3-Phase

PHASE 1 · ML

날씨 예측 모델

과거 날씨 CSV → 내일 기온 / 비 예측
모델 (LR-RF-XGB). scikit-learn.

결과

MAE 2.50°C



PHASE 2 · DL

시계열 딥러닝

같은 데이터, LSTM 으로 정교화. 7 일
시퀀스 → 며칠치 예보. Keras.

결과

MAE 2.41°C 역전



PHASE 3 · LLM/RAG

출근 비서

DL 예측 + 교통 API → 로컬 sLLM 이
자연어 브리핑 → 디스코드 알림. Ollama
EXAONE 3.5

결과

END-TO-END 자동 발송

→ 전 단계 결과물이 다음 단계의 입력 자료로 사용됨 — 점진적 확장 전략.

02

PHASE 1 · MACHINE LEARNING

Phase 1 — ML: 날씨 예측 모델

scikit-learn 으로 회귀 / 분류 기본기 다지기 .

{ REGRESSION

▣ CLASSIFICATION

🏠 SCIKIT-LEARN

데이터 수집 & EDA

10.5 년 실데이터를 모으고 , ML 입력 · 정답 스키마를 설계한 과정 .

REAL-WORLD KMA DATA



DAY
S

3,836 일

실데이터 (10.5 년)



FEATUR
ES

7 개

피쳐 — 기온 · 습도 · 기압 · 풍속 · 강수량



STATI
ON

서울 (108)

기상청 ASOS 일자료



SCALE-
UP

2.5 년 → 10.5
년

데이터 확장 — DL 역전의 결정적 계기

데이터 파이프라인

- 기상자료개방포털 `data.kma.go.kr` 수동 다운로드
- `import_kma.py` 로 EUC-KR CSV 를 스키마 변환
- 합성 데이터 → 실데이터로 교체 (풀백 유지)

입력 · 정답 설계

- 입력 X: 오늘 기온 · 습도 · 기압 · 어제 강수량
- 정답 y: 내일 기온 (회귀) · 비 여부 (분류)
- `train_test_split` 으로 시계열 순서 보존

EDA — 피쳐 상관관계 분석

수치형 7 개 피쳐 간 Pearson 상관관계 히트맵 — 타깃 설계와 다중공선성
 제거

SEABORN · COOLWARM

주요 인사이트

- temp_avg·temp_max·temp_min: 거의 완벽한 양의 상관 (≥ 0.95) — 일교차 패턴 안정
- pressure ↔ temp_avg: 음의 상관 (~ -0.7) — 기온 ↑ 기압 ↓ 물리패턴
- humidity ↔ precip: 양의 상관 — 비 오는 날 습도 ↑ 검증
- wind_speed: 타 피쳐와 낮은 상관 (≤ 0.3) — 독립 정보원

설계 결정

temp_max/min 타깃은 일교차 일관성 확보, 풍속은 분류 모델에서 정보 이득 기대.

FEATURES · 7 × 7

Pearson r

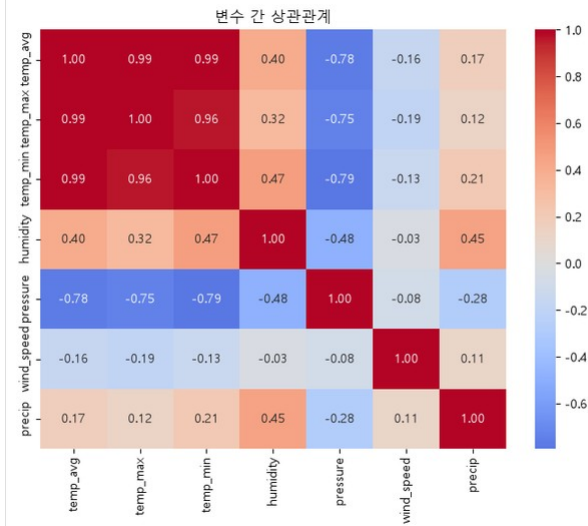


그림 1 · 수치형 7 개 피쳐 상관관계 히트맵 (seaborn, coolwarm, 중심값 0)

모델 3 종 비교 & 잔차 분석

쉬운 모델부터 점진 적용 — LinearRegression → RandomForest → XGBoost

RESULT · 회귀 MAE(°C)

3 models · 3,836 days

모델	회귀 MAE(°C)	비고
LinearRegression	2.50	⚡최고 (회귀)
RandomForest	2.6x	중간
XGBoost	2.7x	과적합 경향

잔차 분석 인사이트

잔차 분석으로 모델별 편향 패턴 확인 — 큰 편차 없이 무작위 분포 → 모델이 체계적 오차 없이 학습됨.

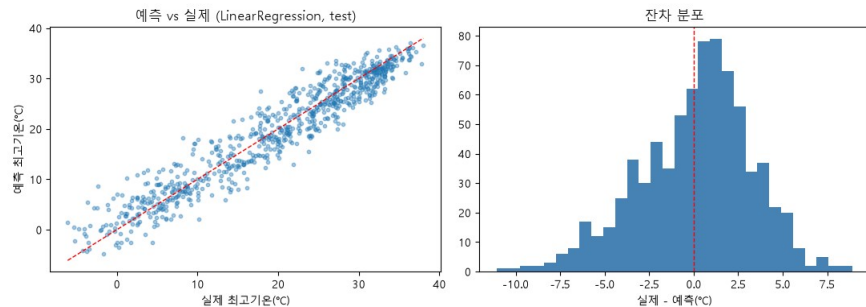


그림 2 · 잔차 분포 — 예측 오차가 0 주변에 무작위 분포함을 확인

ML 평가 심화 — 분류 성능 & 피처 중요도

✓ CROSS-VALIDATION

✓ 회귀 MAE 만 보지 않고 분류 성능 (혼동행렬) 과 피처 기여도 (중요도) 로 모델 신뢰성 교차 검증.

CONFUSION MATRIX

LogisticRegression · Binary

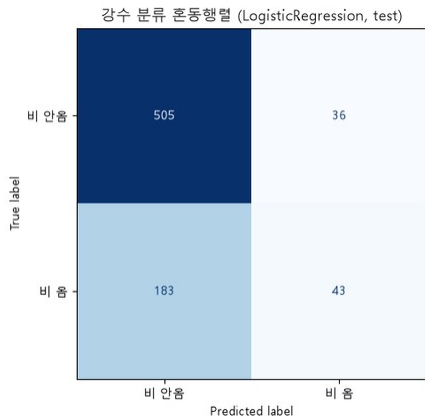


그림 3 · 혼동행렬 — LogisticRegression 강수 이진분류 (Accuracy 0.71)

TAKEAWAY TN·TP 비율 높음, FN(비를 못 맞춘) 이 주요 오차 — 강수 분류는 회귀보다 어려움 확인

FEATURE IMPORTANCE

RandomForest · Gini

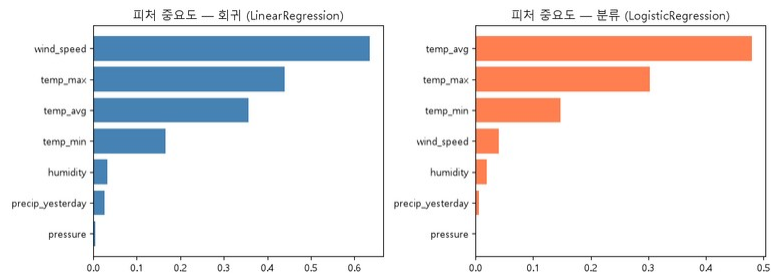


그림 4 · 피처 중요도 — RandomForest 기준, 타깃 예측 기여도

TAKEAWAY 어제 강수량 · 현재 습도가 상위 — 직관과 일치. 풍속 · 기압은 낮지만 분류엔 기여

PHASE 2 · DEEP LEARNING

Phase 2 — DL:

시계열 딥러닝

LSTM 으로 순서를 기억하게 만들기 .

{ TIME-SERIES

STM (SEQ= 7)

◆ KERAS

03

/ 04

PHASE

LSTM 도입 배경 & 학습 구조

시계열 흐름을 학습하기 위해 단층 LSTM + 멀티헤드 출력 구조를 채택

ERAS · LSTM(32)

왜 DL 인가?

날씨는 순서가 중요 — 어제 · 그제 흐름이 내일에 영향 · ML 은 순서를 잘 못 보지만 LSTM 은 순서를 기억한다.

LSTM 모델 구조

- + 입력 · 지난 7 일 시퀀스 SEQ_LEN= 7
- + 정규화 · MinMaxScaler
- + 레이어 · LSTM(32) 단층 + 멀티헤드 출력
- + 출력 · 회귀 2 출력 (최고 / 최저기온) + 분류 1 출력 (비 여부)
- + 손실 · MSE + binary_crossentropy 가중합

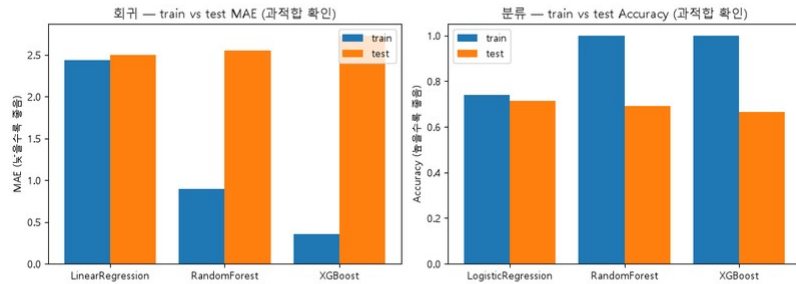


그림 5 · 학습 / 검증 손실 곡선 — 과적합 없이 수렴 확인

ML vs DL 정확도 비교

같은 데이터로 학습한 ML(LR) 과 LSTM 의 회귀 · 분류 성능을 정량 비교

↩ 0.5 년 데이터

회귀 MAE (°C) · 낮을수록 좋음

↘ LSTM 역전

LSTM

2.41 °C

VS

ML (LR)

2.50 °C

Δ -0.09

분류 ACCURACY · 높을수록 좋음

≡ 거의 동률

LSTM

0.706

VS

ML

0.714

Δ -0.008

KEY INSIGHT · 핵심 학습 포인트

데이터 2.5 년 → 10.5 년 확장 후 LSTM 이 우위 · DL 은 데이터 충분해야 진가 발휘 — 핵심 학습 포인트.

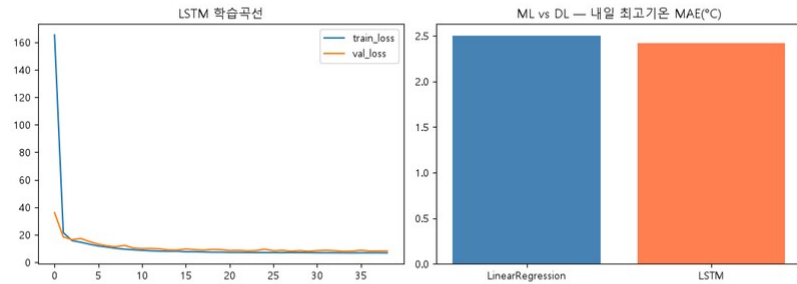


그림 6 · ML vs DL 예측 오차 비교 — LSTM 이 ML 을 근소하게 역전

시계열 예측 결과 — 며칠치 예보를 한 번에

LSTM 모델이 7 일 시퀀스를 입력받아 향후 며칠의 기온 흐름을 한 번에 예측.

{ LSTM FORECAST

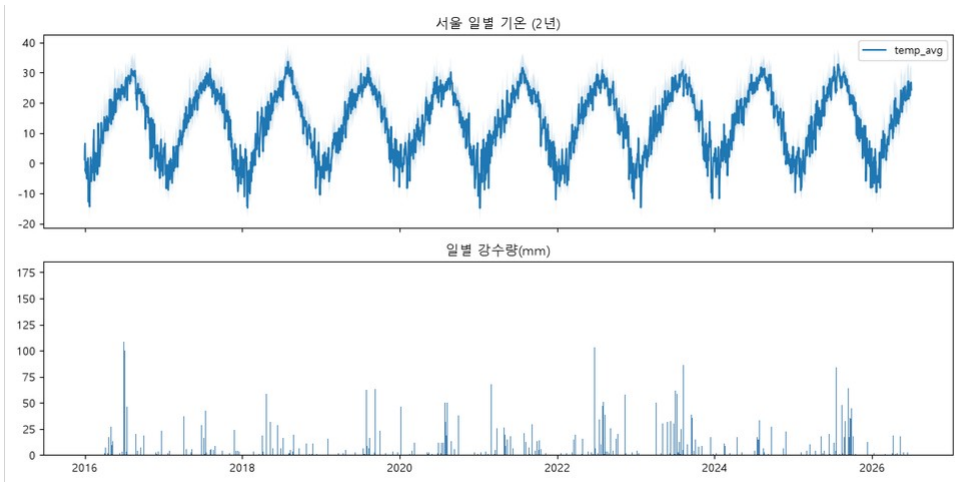


FIGURE 7 시계열 예측 — 과거 7 일 시퀀스로 향후 예측, 실측값과 비교

1

✓ INSIGHT

충분한 데이터 → DL 우위 확인

2

↓ REUSE

LSTM 모델 .keras 저장·재사용

3

↗ HANDOFF

contract.md 스키마로 Track B에 전달
(source: lstm)

04

/ 04

CHAPTER 04

Phase 3 — LLM / RAG: 출근 비서

예측을 사람 말로 번역하고,
매일 아침 폰으로 보낸다.

DL 예측



RAG



SLLM



Discord

- FINAL PHASE

RAG 파이프라인 & 로컬 sLLM 전환

PIPELINE · 5 STEPS



CHALLENGE 01

RAG 임베딩 전환

- Chroma 기본 ONNX 모델 (79MB) 다운로드 반복 timeout
- TF-IDF(sklearn) 로 교체 — 문서 6~10 개 규모라 매 검색마다 재구축해도 즉시 처리
- 오검색 방지 : 니치 문서는 별도 폴더, 1:1 직접 읽기

CHALLENGE 02

sLLM 백엔드 전환

- Claude API → 로컬 Ollama EXAONE 3.5 7.8B (4.8GB, 한국어 특화)
- 선생님 과제 취지 : 외부 API 대신 sLLM 직접 운용 학습
- 분류 : temperature=0 (결정적), 생성 : 기본값
- RTX 4060 8GB 에서 8~12 초 응답, keep_alive=30m 으로 상시 로드

디스코드 봇 & 차별화 기능

대화형 디스코드 봇 — 자연어 의도 분류 15+ **인텐트** 로 모든 기능을 한 채널에서 처리

SIGNATURE

01 · NICHE



니치 타겟 · 자전거 통근

별도 지식 폴더에서 1:1 직접 읽기 전용 섹션으로 분리 생성 버튼 또는 채팅으로
토글

CONFIG

02 · ROUTINE



출근지 설정

출발지 · 도착지 한 번 등록 → 이후 알림에 날씨 + 소요시간 같이 발송 자전거 니치일
때 자동 생략

PRD CORE

03 · FEEDBACK



개인 체감 피드백 루프

매일 알림에 🧠 / 👍 / 🧐 버튼 누적 평균으로 체감온도 보정치 (°C) 계산 브리핑에
자동 반영

AUTOMATION

04 · UX



자동 알림 & UX

자연어로 시간 설정 (매일 8 시에 알려줘) @ 멘션으로 푸시 보장 도움말 버튼 패널
자동 핀 고정

코드 아키텍처 — 20 개 모듈의 책임 분리

데이터 → 예측 → 서비스 3 계층 흐름과 Track A·B·인프라·테스트 4 축 모듈

인벤토리
DATA FLOW · 3 LAYERS

0 MODULES



MODULE INVENTORY · 4 TRACKS x 5

Track AML / DL	Track BRAG / LLM	Infra 인프라	Test 테스트
import_kma / generate_data	rag.py (Chroma+TF-IDF)	app.py (Streamlit)	test_predict.py
eda.py / train.py	llm.py (Ollama EXAONE)	discord_bot.py (웹훅)	test_dl_predict.py
ml/predict.py	briefing.py (브리핑)	discord_chat_bot.py	(self-check)
dl/predict.py	traffic.py (TMAP/ODsay)	seed_feedback.py	predictor.py __main__
dl/sequences.py	feedback.py (체감 학습)	predictor.py (파사드)	—

◆ predictor.py = Track A·B 결합도 분리 핵심 — Track B 는 ML/DL 내부 로직을 몰라도 get_prediction() 한 번으로 예측값 획득 .

코드 리뷰 ① — ML/DL 예측 모듈

Track A 두 축 — scikit-learn 회귀 / 분류 파이프라인과 Keras LSTM 시계열 변환의 핵심 설계 포인트

2 MODULES · 6 CODE POINTS



src/ml/predict.py

데이터 → 피처엔지니어링 → 3 종 모델 학습 / 비교 → 예측, 전부 포함

TRACK A · ML

시계열 분할

CP
01

`train_test_split(test_size=0.2, shuffle=False)` — 시간순 앞 80% / 뒤 20% 분리로 미래 데이터 누수 방지. 랜덤 셔플 시 시계열에 부적절.

배포 모델 = 전체 재학습

CP
02

평가는 분리된 `test` 로, 배포는 `best` 모델 종류로 `train+test` 다시 합쳐 재학습. 평가와 배포가 다른 데이터를 쓰는 의도적 설계.

contract 준수

CP
03

`predict_tomorrow()` 반환 = `{date, temp_max, temp_min, rain_prob, source}`. `source` 는 최고 회귀 모델명 소문자 — Track B 가 모델 종류를 알 수 있음.



src/dl/{sequences, predict}.py

LSTM 시계열 변환 — 슬라이딩 윈도우 생성 + 정규화 + 멀티헤드 출력

TRACK A · DL

슬라이딩 윈도우

CP
01

`SEQ_LEN=7` — 최근 7 일을 `x`, 그 다음날을 `y`로 묶어 3 차원 텐서 `(N, 7, 7)` 생성. 시계열을 supervised learning 형태로 변환.

정규화 = 전체 `fit` (data leakage 감수)

CP 02 · KEY

`MinMaxScaler` 를 전체 데이터에 `fit` — 914 행 데이터에 약한 leakage 감수. ponytail 주석: 데이터 대폭 늘면 `train` 만 `fit` 하도록 분리해야 한다고 명시 (알려진 한계 + 업그레이드 조건 기록).

재학습 폴백

CP
03

`try/except` 로 모델 로드 실패 시 그 자리에서 재학습 — Keras 버전 불일치로 저장 / 로드 호환 깨지는 문제 구조적 회피.

공통 인터페이스 `load_models()` → `predict_tomorrow()` — `app.py.predictor.py` 가 두 트랙을 동일 코드로 다룰 수 있음.

코드 리뷰 ② — RAG + sLLM 브리핑

Track B — 검색 + 로컬 sLLM 호출, 그리고 두 축을 결합해 근거 인용 브리핑을 만드는 파이프라인

2 MODULES · 7 CODE POINTS

src/{rag, llm}.py
 검색 + 로컬 sLLM 호출 — Chroma→TF-IDF 교체, Ollama TRACK B · RETRIEVAL
 EXAONE 3.5 7.8B

TF-IDF 임베딩 교체

CP 01 · KEY

Chroma 기본 ONNX(79MB) 다운로드 반복 timeout → sklearn TfidfVectorizer 로 교체. 문서 6~10 개라 매 검색마다 컬렉션 재구축해도 즉시 처리. ponytail: 지식베이스 커지면 실제 임베딩 모델로 교체 필요 명시.

매 호출 재구축

CP 02

delete_collection 후 재생성 — 문서가 바뀌어도 항상 최신 상태 유지. 단, 검색마다 재구축 비용 발생 (문서 수 적어 감수).

temperature=0 분류

CP 03

Ollama EXAONE 3.5 7.8B. 분류는 temperature=0 (결정적), 생성은 기본 0.7. keep_alive=30m 으로 재로딩 (수십 초 지연) 방지. 실험 근거 docstring에 명시.

src/briefing.py
 RAG+LLM 결합 브리핑 — 결정적 검색어 + 근거 인용 강제 + TRACK B · BRIEF
 개인화 주입

결정적 검색어 구성

CP 01

_build_query() — rain_prob ≥ 0.4 면 우산, 일교차 ≥ 8 도면 옷차림 등 규칙 기반 검색어. LLM 에게 자유 질의 맡기지 않아 검색 결과 일관성 / 설명가능성 확보.

근거 인용 강제

CP 02 · KEY

프롬프트에 근거 문서를 [문서명] 으로 표시하도록 지시 — RAG 의 확인 가능성 요구 충족. CLAUDE.md 의 RAG 브리핑은 컷 금지 항목.

니치 = 검색 안 함

CP 03

generate_niche_briefing() — 자전거 니치는 검색 없이 1:1 문서 직접 read. (1) 결정적 매핑이라 검색 불필요, (2) 일반 풀과 섞이면 오염 방지.

개인화 주입

CP 04

personal_offset(체감 보정치) 를 프롬프트에 추가 — 이 사용자는 N°C 더 춥게 / 덥게 느낀다는 문장으로 옷차림 조언에 반영.

🌱 RAG 파이프라인 = 검색(rag.search) + 생성(llm.chat) 결합 — 근거 인용 프롬프트로 환각 최소화.

코드 리뷰 ③ — 디스코드 봇 & 피드백 학습

메인 실행 앱과 보조 모듈 — UX 지연 최소화 전략과 체감 학습의 의도적 단순화

3 MODULES · 8 CODE POINTS

src/discord_chat_bot.py MAIN · BOT
메인 실행 앱 — sLLM 분류 + 프리페치 캐싱 + 버튼 질약 + defer 패턴

LLM 의도 분류

CP 01

classify_message() — 키워드 매칭 아님. 로컬 EXAONE 이 자연어 전체를 11 개 라벨로 분류 (temperature=0). HELP 는 최우선 규칙, SCHEDULE 은 매일 없어도 시각 + 알람이면 분류.

정각 3 분 전 프리페치

CP

PREP_MINUTES=3 — 정각 3 분 전에 미리 브리핑 생성해 캐시. 정각에 LLM 호출 (8~12 초) 지연을 발송 시점에서 숨기는 전략. _settings_snapshot() 으로 설정 변경 시 캐시 무효화.

버튼 = LLM 호출 질약

CP

SettingsView/FeedbackView — 선택지 정해진 기능 (니치 토글, 피드백) 은 discord.ui 버튼으로 딸각 처리. LLM 분류 호출 안 함. timeout=None+ 고정 custom_id 로 재시작 후에도 버튼 동작.

defer 패턴

CP

clear_chat — purge() 3 초 넘게 걸리면 인터랙션 토큰 만료 (404). defer() 먼저 호출 후 followup.send() 로 응답 — 긴 작업의 디스코드 표준 패턴.

src/{feedback,traffic}.py SUPPORT · DATA
체감 학습 + 교통 — 라벨 평균 보정과 geocode 통일, 명시적 실패

라벨 평균 보정 (회귀 x)

CP 01

get_personal_offset() — cold=+2.5°C, good=0, hot=-2.5°C 라벨별 평균. 표본 3 개 미만이면 None (중립). 회귀 모델 대신 단순 평균 — 표본 적으면 회귀 과적합으로 신뢰성 저하 (ponytail 판단).

save_last_weather

CP

피드백 버튼 눌렀을 때 그날 날씨를 알아야 하므로 발송 시점마다 채널별 마지막 날씨 기록. 없으면 record_feedback() 이 False 반환 — 발송 이력 없는 채널의 모순 상태 방지.

geocode 통일

CP

traffic.py — 자차 (TMAP)·대중교통 (ODsay) 모두 좌표 필요해 geocode() 하나로 통일 재사용. 환경변수 없으면 즉시 KeyError — 조용한 실패 방지.

명시적 실패

CP

ODsay 경로 없으면 {minutes:None, error: 경로 없음} 반환 — 예외 대신 명시적 실패. 호출부가 None 을 보고 안내 메시지로 분기.

◆ 메인 앱 = 상시 루프 (1 초 주기) + sLLM 분류 + 버튼 질약 + 프리페치 캐싱 — 사용자 경험 지연 최소화가 핵심.

문제해결 스토리 & 핵심 성과

데이터 · 임베딩 · sLLM · 배포 4 개 디버깅 사가 모델 · 시스템 신뢰성을 만들었고, 4 개 지표가 end-to-end 완성을 증명한다.

🔧 문제해결 4 선

DEBUG
STORIES

01 데이터

합성 데이터 → 실데이터 교체 · 2.5 년 → 10.5 년 확장으로 LSTM 우위 확인

02 임베딩

Chroma ONNX 다운로드 timeout → TF-IDF 로 교체, 문서 수 적어 즉시 처리

03 sLLM

의도 분류 비결정적 → temperature=0 고정으로 완전 일치

04 배포

Discord Cloudflare urlib 차단 → User-Agent 헤더 추가로 해결

🔧 DEBUG · 4 METRICS

👑 핵심 성과

KEY
METRICS

2.41 °C

회귀 MAE
LSTM 최고 성능



15 +

인텐트
로컬 sLLM 의도 분류



end-to-end

검증
DL → RAG → sLLM → 디스코드 자동 발송



4 /4

차별화
교통 · 출근지 · 니치 · 피드백 전부 구현 완료



트러블슈팅 — 디스코드 봇 도움말 중복 발송

백그라운드 재실행 후 사용자 메시지 하나에 도움말이 2 번씩 발송되던 현상 📖 2026-

◆ ISSUE · 4 STEPS

01 / 04

● 상황

봇 백그라운드 재실행 후 사용자 메시지 하나에 도움말·모든 응답이 2 번씩 나옴.
사용자 경험 크게 저하.



02 / 04

⚠ 원인

봇 프로세스가 2 개 동시 실행 — 각각 Discord Gateway 에 로그인해 메시지마다 두 인스턴스가 각각 응답. 1 차 실행 (PID 1565) 이 sleep+ 로그 확인
무음으로 백그라운드에 남아있었고, ps aux 로 인자 안 보여 죽은 걸로 착각해 2 차 실행 (PID 2281) 추가.



03 / 04

💡 해결

ps -ef (전체 명령행 표시) 로 재확인
— -m src.discord_chat_bot
프로세스 2 개 (1565, 2281) 확인.
오래된 쪽 (1565) 종료, 최신 로그인된 2281 만 유지. 중복 응답 즉시 사라짐.



04 / 04

🛡 재발방지

Windows/MSYS 환경에선 ps aux 가 아니라 ps -ef 로 명령행까지 확인.
봇처럼 상시 실행 프로세스는 재실행 전 반드시 기존 인스턴스부터 ps -ef | grep 으로 확인.



핵심 교훈 · KEY LESSON

백그라운드 프로세스 생존 확인은 ps aux 가 아니라 ps -ef — 인자까지 봐야 오답 방지. 디버깅 도구의 한계를 아는 것 자체가 디버깅 실력이다.

LESSON · 01

2026 · 학습 프로젝트 회고

감사합니다.

ML → DL → LLM/RAG · 개인형 출근 비서

LESSON 01



학습 여정

ML→DL→LLM을 한 테마로 연결하니 각 단계가 자연스럽게 다음 단계의 재료가 됨

LESSON 02



데이터의 힘

DL은 데이터 충분해야 진가. 10.5년 확장이 결정적

LESSON 03



sLLM 운용

로컬에서 끝까지 돌려보니 비용·지연·프라이버시 이점 체감

→ NEXT

캘린더 연동 · 니치 확장 (배달 라이더) · 옷차림
캐릭터 추천

